🔥 ONE FULL DEVOPS PROJECT (END-TO-END)
Non-IT fresher friendly | Real-time | Interview ready
🧩 Project Title
CI/CD Pipeline to deploy a backend application using
Git + Jenkins + Docker + AWS + Kubernetes
🧠 Project aim (simple words)
Developer code push చేస్తే
automatically build → container → deploy → API live అవ్వాలి
🏗️ Architecture (visualize this)
Developer
|
GitHub
|
Jenkins (CI/CD)
|
Docker Image
|
AWS EC2 / EKS
|
Kubernetes Pod
|
LoadBalancer / Service
|
API Access
🍼 Application (Don’t panic 😄)

👉 Simple sample backend app
Spring Boot or NodeJS
Already available code
You DON’T write logic
🔧 TOOLS USED

Area	Tool
OS	Linux
SCM	GitHub
CI/CD	Jenkins
Container	Docker
Orchestration	Kubernetes
Cloud	AWS
API Test	Postman

🚶 STEP-BY-STEP FLOW (THIS IS GOLD)
🔹 STEP 1: Developer pushes code
Code in GitHub
Has:
Source code
Dockerfile

👉 DevOps role starts here
🔹 STEP 2: Jenkins pipeline triggers
Pipeline stages:
1️⃣ Checkout code
2️⃣ Build app
3️⃣ Create Docker image
4️⃣ Push image
5️⃣ Deploy

👉 If build fail → check logs
🔹 STEP 3: Build stage
Examples:
Java → mvn clean install
Node → npm install

👉 Output:
.jar file or node_modules
🔹 STEP 4: Docker image creation 🐬
Dockerfile example:
FROM openjdk:17
COPY target/app.jar app.jar
CMD ["java","-jar","app.jar"]

👉 DevOps checks:
Base image
Port
CMD
🔹 STEP 5: Push image to registry
Docker Hub / ECR

👉 Used by Kubernetes later
🔹 STEP 6: Kubernetes deployment ☸️
Files:
deployment.yaml
service.yaml
Deployment:
Creates Pods
Runs containers
Service:
Exposes API
🔹 STEP 7: Access API 🌐
http://<loadbalancer-ip>:8080/health

👉 API UP = SUCCESS 🎉
🧠 WHERE REAL-TIME ISSUES COME (IMPORTANT)
❌ Build fail
Wrong command
Dependency issue
Permissions

👉 Fix in Jenkins
❌ Image build fail
Wrong Dockerfile
Missing file

👉 Fix Dockerfile
❌ Pod crash
Wrong env variable
Port mismatch

👉 kubectl logs
❌ API down
Security group
Service config
App crash

👉 Infra + app check
🎯 HOW TO EXPLAIN IN INTERVIEW (MAGIC ANSWER)
"We have a CI/CD pipeline where developer pushes code to GitHub.
Jenkins triggers build, creates Docker image, pushes it to registry,
and deploys it to Kubernetes on AWS.
We monitor logs and handle failures at build, container, or API level."

👆 This answer alone impresses interviewer.
🪫 Resume point (copy this later 😉)

Implemented end-to-end CI/CD pipeline using Jenkins, Docker, Kubernetes on AWS to deploy backend application with automated build, containerization, and monitoring.

🔑 Why this project is enough
✅ Covers 90% DevOps interview questions
✅ Shows real-time understanding
✅ No heavy coding
✅ Fresher friendly

👉 Next step (choose now):
1️⃣ Real-time troubleshooting scenarios (deep practice)
2️⃣ Interview Q&A (Telugu, fresher level)
3️⃣ Week-wise hands-on commands for this project

Number cheppandi 👇
Real-time troubleshooting scenarios (deep practice)
2️⃣ Interview Q&A (Telugu, experience level)
3️⃣ Week-wise hands-on commands for this project

Perfect 👍
ఇప్పుడు నేను DEVOPS ENGINEER TRAINING MODE లోకి వెళ్తున్నాను 😄
మీకు 3 things ని clear + practical + interview-ready గా ఇస్తాను.

👉 Order ఇలా ఉంటుంది (best learning):
1️⃣ Real-time troubleshooting scenarios (DEEP)
2️⃣ Interview Q&A (Telugu – experience style answers)
3️⃣ Week-wise hands-on commands (copy–paste practice)
1️⃣ REAL-TIME TROUBLESHOOTING SCENARIOS (DEEP PRACTICE)

🔥 Scenario 1: Jenkins Build FAIL
Problem: Pipeline red ❌
How DevOps thinks:
Jenkins logs open చేయి
Error first occurrence చూడాలి
Common root causes:
mvn not installed
npm permission issue
Internet blocked
Commands:
java -version
mvn -v
npm -v
df -h
Interview answer:
"First I check Jenkins console logs, identify build stage failure, verify tools and permissions on Jenkins server."

🔥 Scenario 2: Docker Image Build FAIL
Problem: docker build fails
Root causes:
Wrong base image
File not found
COPY path wrong
Commands:
docker build -t myapp .
docker images
Fix:
Dockerfile paths check
Base image match with app

🔥 Scenario 3: Container Starts and Exits
Problem: Container crash loop

Root causes:
App error
No foreground process
Commands:
docker ps -a
docker logs <container-id>
Interview line:
"I check container logs to see if application is crashing or exiting due to config or runtime issues."

🔥 Scenario 4: Kubernetes Pod CrashLoopBackOff
Problem: Pod restarting continuously
Root causes:
Wrong env variables
Port mismatch
Memory issue
Commands:
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>

🔥 Scenario 5: API DOWN in Production
Problem: Users cannot access app
Step-by-step DevOps flow:
Server up?
Pod running?
Logs?
Network?
Load balancer?
Commands:
curl localhost:8080/health
netstat -tulnp
kubectl get svc
Interview style:
"I follow infra → application → logs → network flow to identify root cause."
2️⃣ INTERVIEW Q&A (TELUGU – EXPERIENCE STYLE)
❓ DevOps engineer role ఏమిటి?

👉
"DevOps engineer automates build, deployment, monitoring of applications and ensures system stability."
❓ Build fail అయితే first step?
👉
"I check Jenkins build logs, identify failure stage, and verify environment and permissions."
❓ Container crash అయితే?
👉
"I check container logs, environment variables, and resource limits."
❓ Kubernetes ఎందుకు?
👉
"Kubernetes helps in container orchestration, auto-scaling, and self-healing."
❓ DevOps engineer code రాస్తాడా?
👉
"I don't develop business logic, but I understand application structure to deploy and troubleshoot."
❓ Production issue handle చేశారా? (Fresher trick 😄)
👉
"Yes, during hands-on project, when API was down, I checked pod logs, service configuration, and resolved port mismatch."

3️⃣ WEEK-WISE HANDS-ON COMMANDS (PROJECT PRACTICE)
🗓️ Week 1–2: Linux + Git
ls
cd
tail -f app.log

git clone
git pull

🗓️ Week 3: Jenkins
systemctl start jenkins

Pipeline stages:
Checkout
Build
Docker build

🗓️ Week 4–5: Docker
docker build -t myapp .
docker run -d -p 8080:8080 myapp
docker logs

🗓️ Week 6–7: AWS
ssh ec2-user@ip
sudo yum install docker

🗓️ Week 8–9: Kubernetes
kubectl get pods
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
🧠 FINAL MINDSET (REMEMBER THIS)
“DevOps engineer is not a tool operator,
he is a problem solver.”